

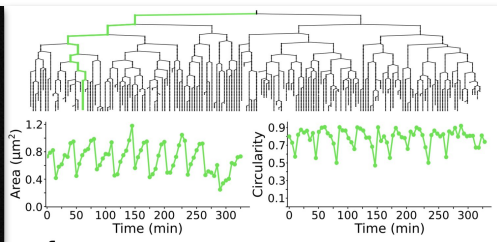
Bio-Image Analysis accelerated by GPU using clesperanto

Stéphane Rigaud - Image Analysis Hub - Institut Pasteur, Paris

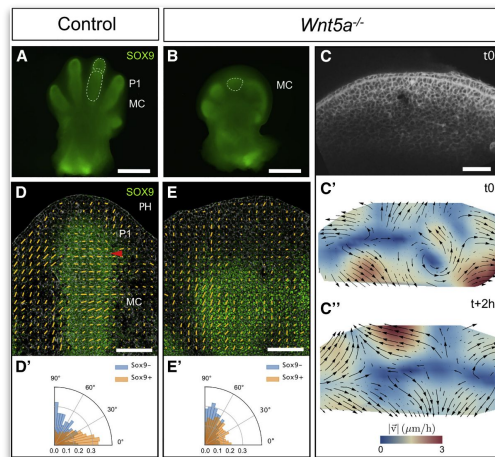
Reusing material from Robert Haase (TUD - PoL)

NEUBIAS Pasteur Course 2025

Bio-Image Analysis



Ershov, Phan, Pylvänäinen, Rigaud, *et.al. nat methods*, 2022



C. Parada, *et.al.*, *Dev Cell*, 2022, Gros Lab

Frameworks



Pipeline

Libraries



Components

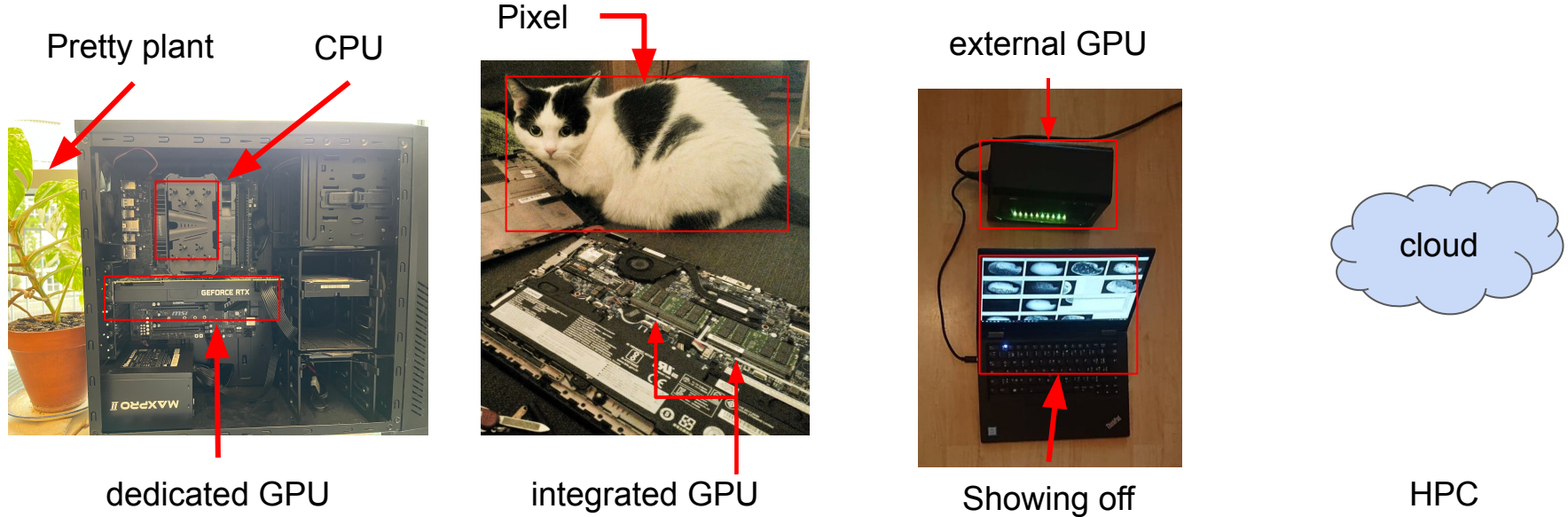
Languages



Written in

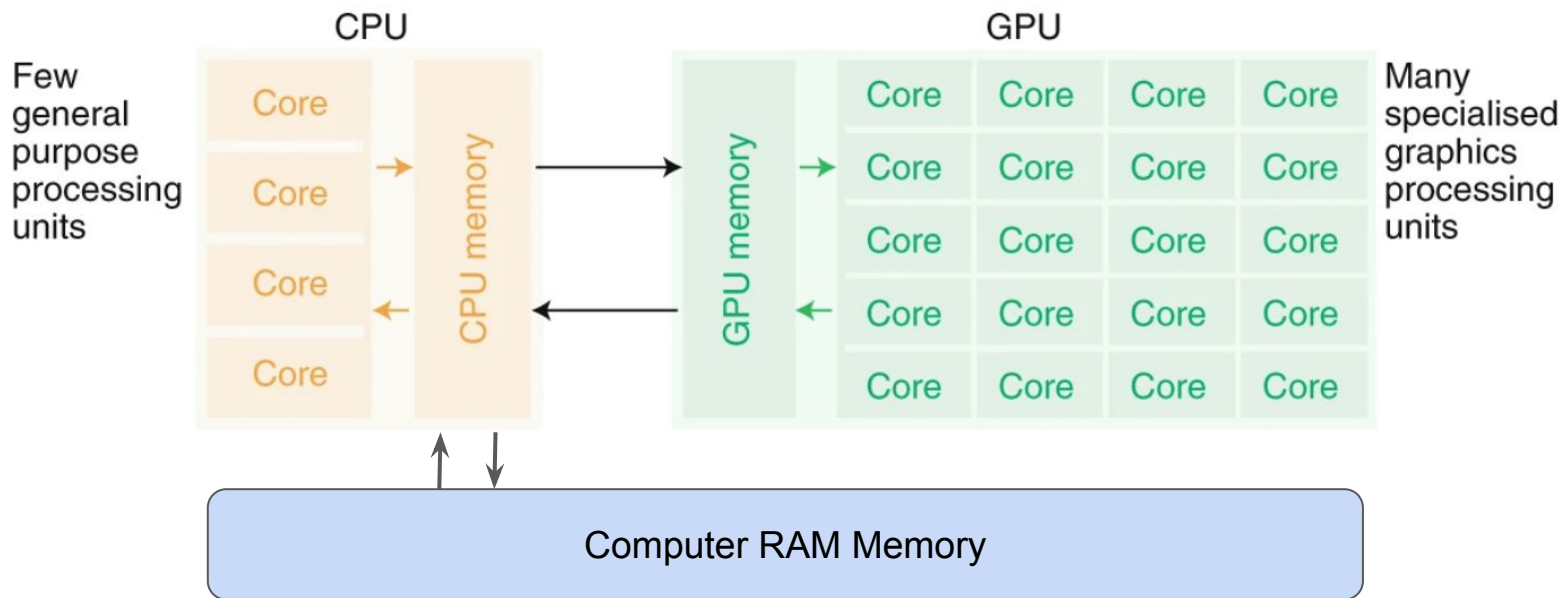
Processing Units

Diversity of units: DSP, TPU, FPGA, etc.



Processing Units

Quality V.S. Quantity

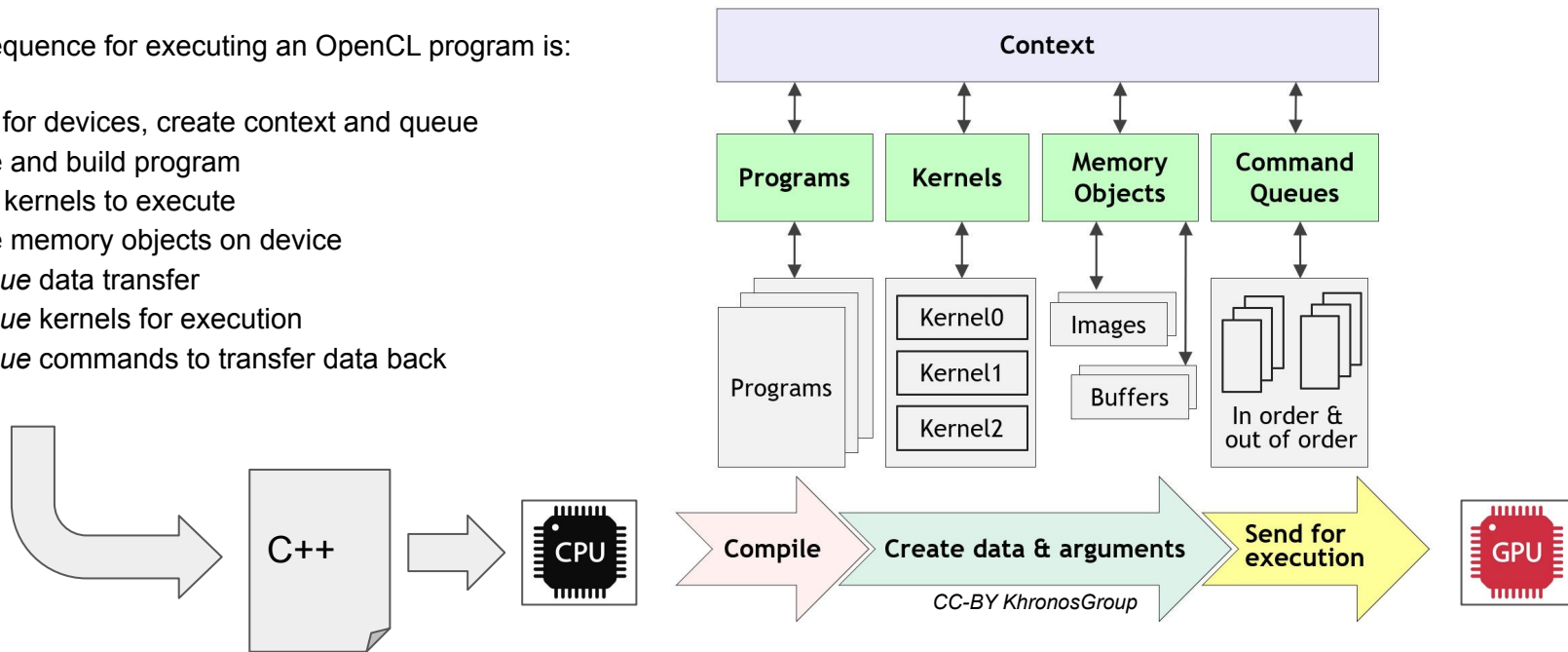


GPU programming

Close to hell

A complete sequence for executing an OpenCL program is:

1. Query for devices, create context and queue
2. Create and build program
3. Select kernels to execute
4. Create memory objects on device
5. *Enqueue* data transfer
6. *Enqueue* kernels for execution
7. *Enqueue* commands to transfer data back



GPU programming

Close to hell ... times 150

```
≡ maximum_z_projection.cl ×
kernels > ≡ maximum_z_projection.cl
You, 18 months ago | 2 authors (You and others)
1  __constant sampler_t sampler = CLK_NORMALIZED_COORDS_FALSE | CLK_ADDRESS_CLAMP_TO_EDGE | CLK_FILTER_NEAREST;
2
3  kernel void maximum_z_projection(
4      IMAGE_src_TYPE src,
5      IMAGE_dst_TYPE dst
6  )
7  {
8      const int x = get_global_id(0);
9      const int y = get_global_id(1);
10
11      IMAGE_src_PIXEL_TYPE max = 0;
12      for (int z = 0; z < GET_IMAGE_DEPTH(src); ++z)
13      {
14          const IMAGE_src_PIXEL_TYPE value = READ_IMAGE(src, sampler, POS_src_INSTANCE(x,y,z,0)).x;
15          if (value > max || z == 0) {
16              max = value;
17          }
18      }
19
20      WRITE_IMAGE(dst, POS_dst_INSTANCE(x,y,0,0), CONVERT_dst_PIXEL_TYPE(max));
21  }
```

User-friendly GPU extension for FIJI



```

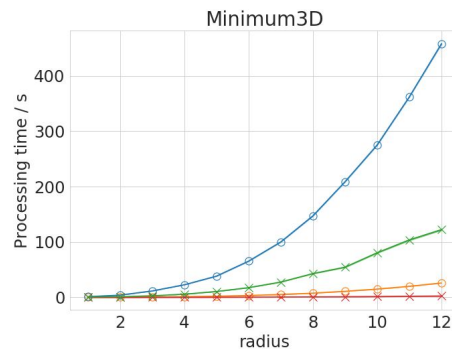
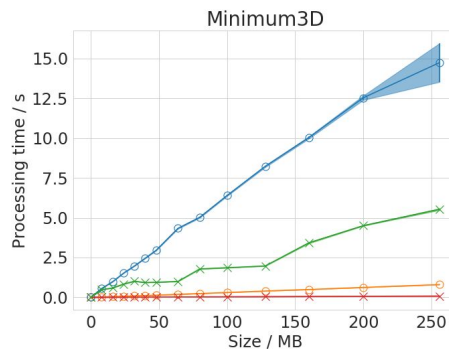
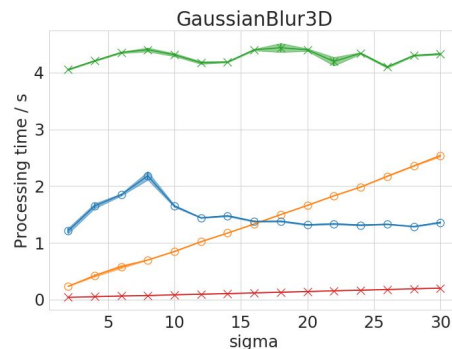
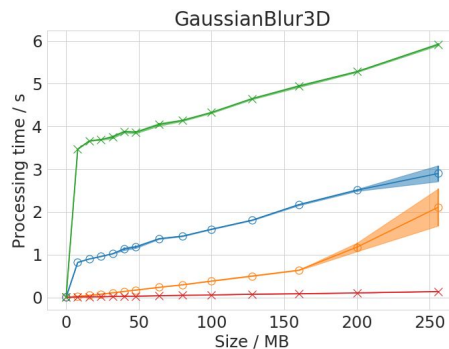
Outline
new1.ijm
File Explorer
+ | File |
stephane
1 // To make this script run in Fiji, please activate
2 // the clij and clij2 update sites in your Fiji
3 // installation. Read more: https://clij.github.io
4
5 // Generator version: 2.5.1.5
6
7 // Init GPU
8 run("CLIJ2 Macro Extensions", "c_l_device");
9
10 // Load image from disc
11 open("/var/folders/d/bb9d0d46v8r3d25p_t98_00000gn/T/temp1693117583914.tif");
12 image_3 = getTitle();
13 Ext.CLIJ2_pushCurrentZStack(image_1);
14
15 // Copy
16 Ext.CLIJ2_copy(image_1, image_2);
17 Ext.CLIJ2_release(image_1);
18
19 Ext.CLIJ2_pull(image_2);
20
21 // Gaussian Blur
22 sigma_x = 2;
23 sigma_y = 2;
24 sigma_z = 2;
25 Ext.CLIJ2_imageJ2GaussianBlur(image_2, image_3, sigma_x, sigma_y, sigma_z);
26 Ext.CLIJ2_release(image_2);
27
28 Ext.CLIJ2_pull(image_3);
29
30 // Threshold Otsu
31 Ext.CLIJ2_thresholdOtsu(image_3, image_4);
32 Ext.CLIJ2_release(image_3);
33
34 Ext.CLIJ2_pull(image_4);
35
36 // Connected Components Labeling Box
37 Ext.CLIJ2_connectedComponentLabelingBox(image_4, image_5);
38 Ext.CLIJ2_release(image_4);
39
40 Ext.CLIJ2_pull(image_5);
41 run("glasbey_on_dark");
42 Ext.CLIJ2_release(image_5);
43
Run Batch REPL Show Errors Clear
Active language: None
Active language: ImageJ Macro
Autocompletion: SciJava supported triggered by Ctrl+Space & auto-display
    
```



Benchmarking



- Workstation CPU
- Workstation GPU
- Laptop GPU
- Laptop CPU



Clesperanto project

OpenCL - Esperanto



- Target multiple languages and frameworks
- Clear and standardized operation call

```
cle.operation_name( input, output, arguments )
```

- Maintain a common API usage between languages
- Reduce implementation redundancy and numerical inconsistency
- Improve code sharing and reproducibility
- Improve maintainability

Clesperanto project

OpenCL - Esperanto

Fiji jython

Python

C++

my_code.x

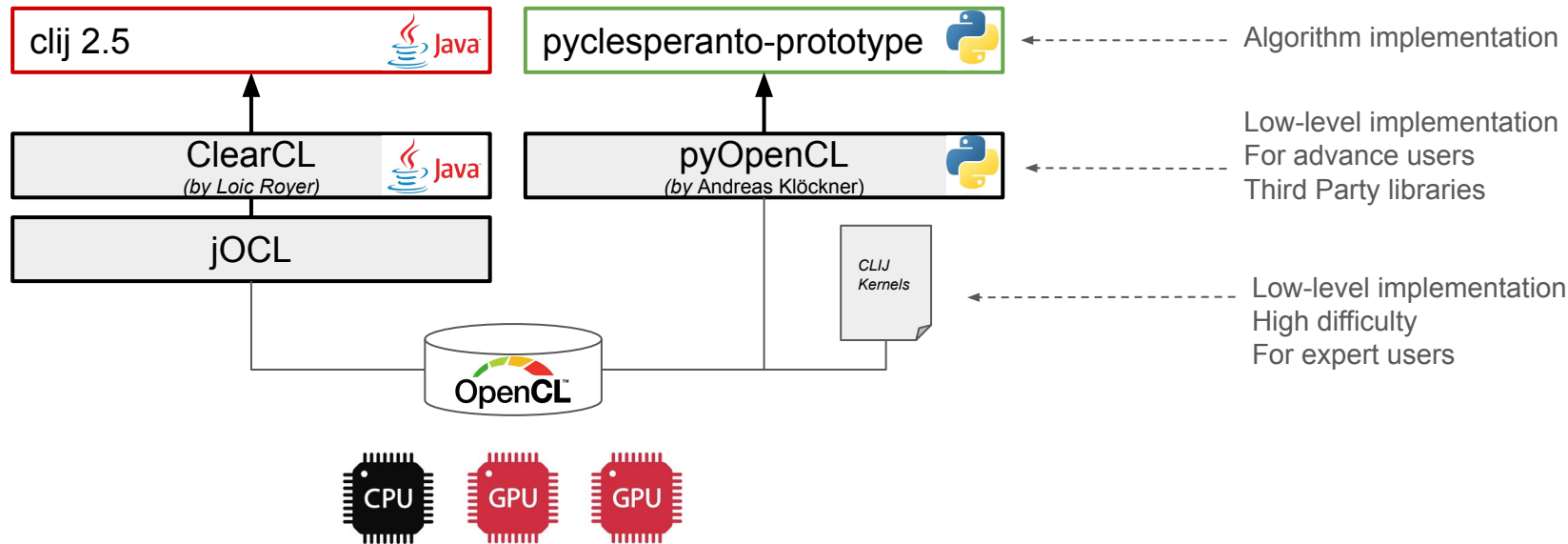
```
auto blurred = cle.create_like(blobs);  
cle.gaussian_blur(blobs, blurred, 3);  
  
auto binary = cle.create_like(blurred);  
cle.threshold_otsu(blurred, binary);  
  
auto label = cle.create_like(binary);  
cle.connect_components_labeling(binary, label);
```

With some
limitations ...

Clesperanto project

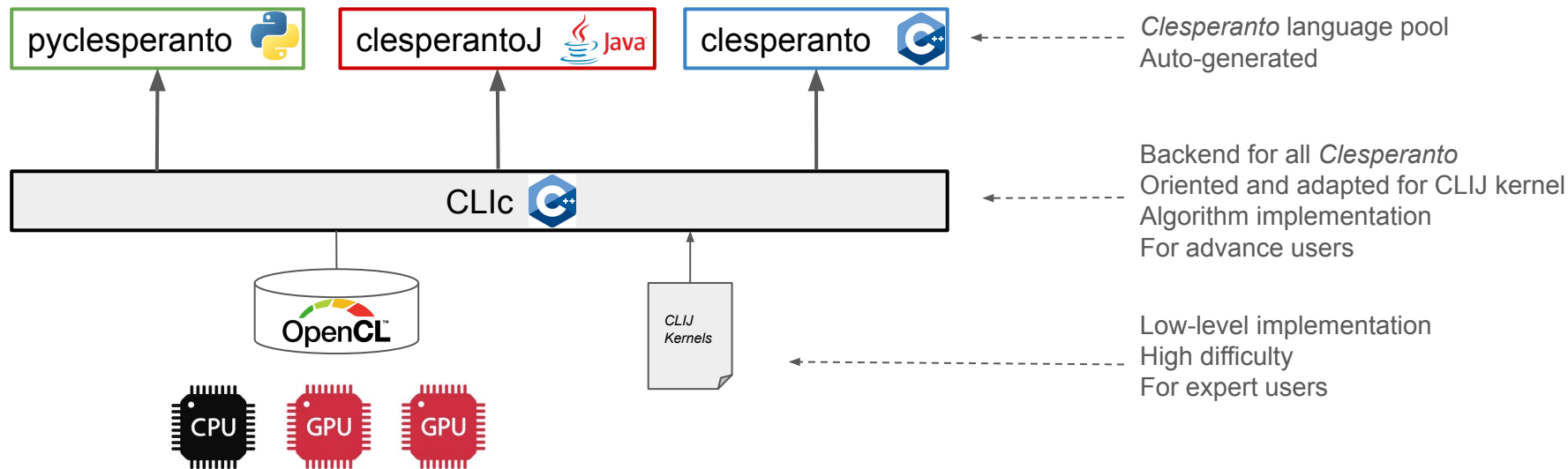
[clearcl](#) | [pyopencl](#) | [jocl](#)

The Prototype



Clesperanto project

Unified architecture, with C under the hood



Practical 1: Introduction & basics

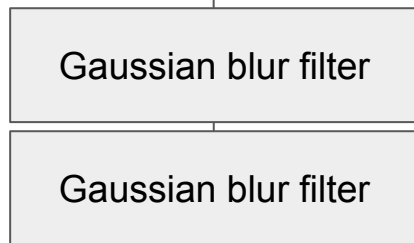
In the Day5 folder, start the environment and jupyter lab

- Find your laptop devices and devices information
- Allocate some data memory to the device
- Push it to the limit!

(optional for NVIDIA) Same with Cupy

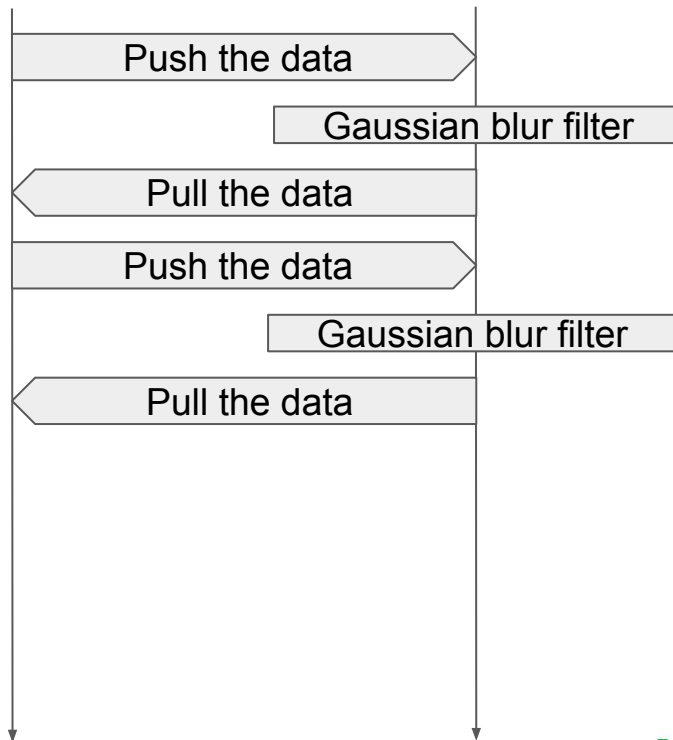
Memory management & workflow construction

Central Processing Unit (CPU)

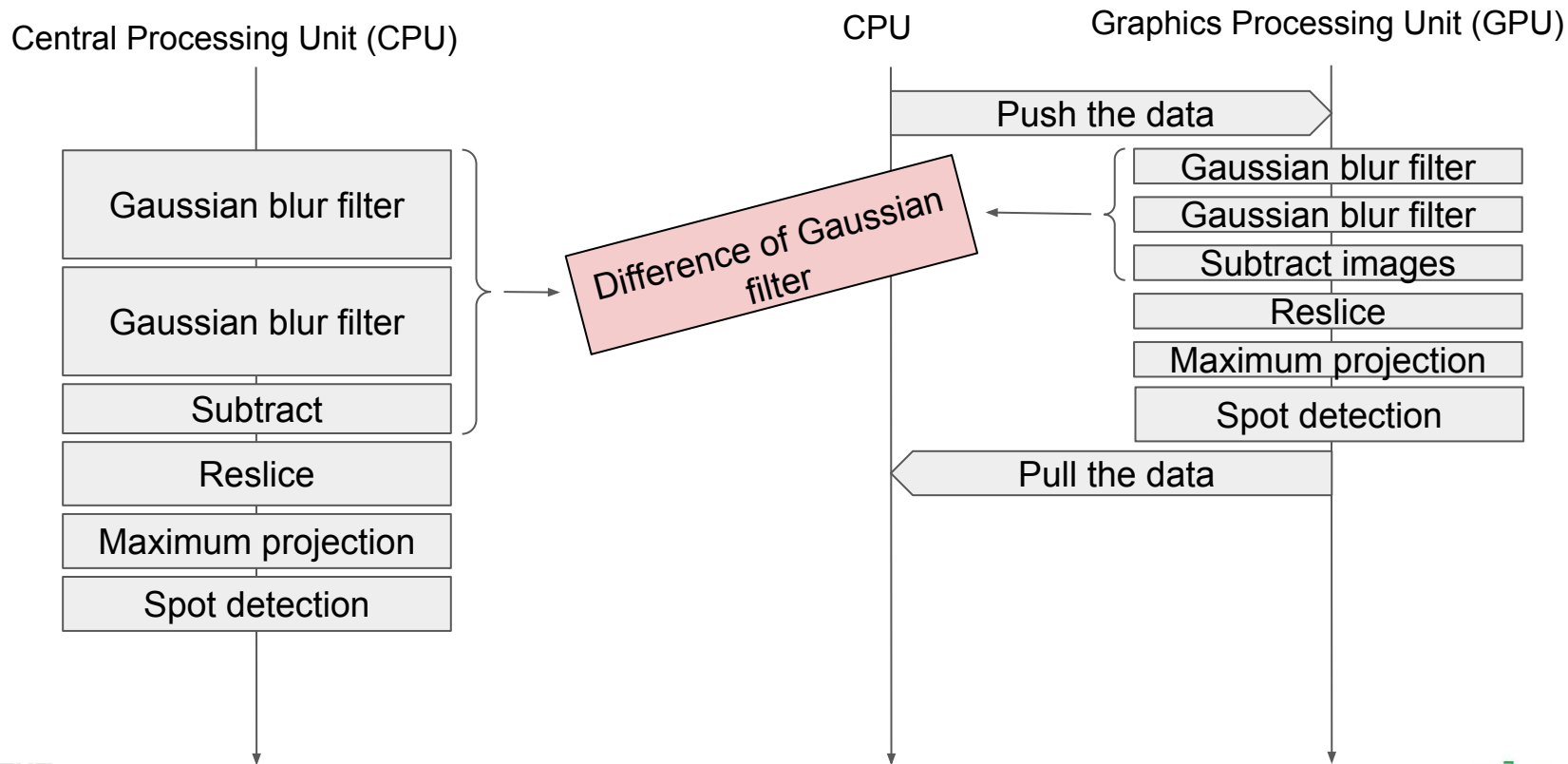


CPU

Graphics Processing Unit (GPU)



Workflow construction & workflow construction



Memory management

Classical way of processing data

Load data

Load data

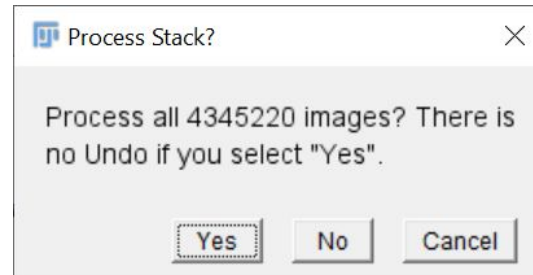
Load data

Load data

Load data

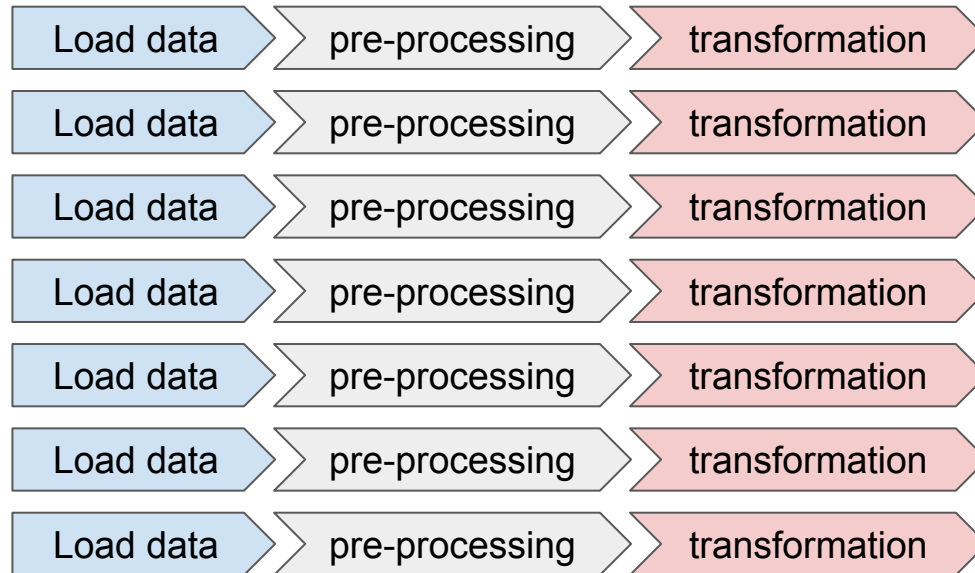
Load data

Load data



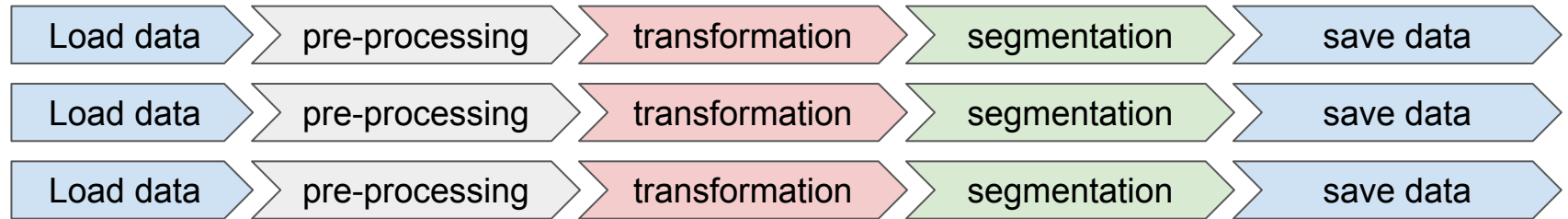
Memory management

Classical way of processing data ... is suboptimal



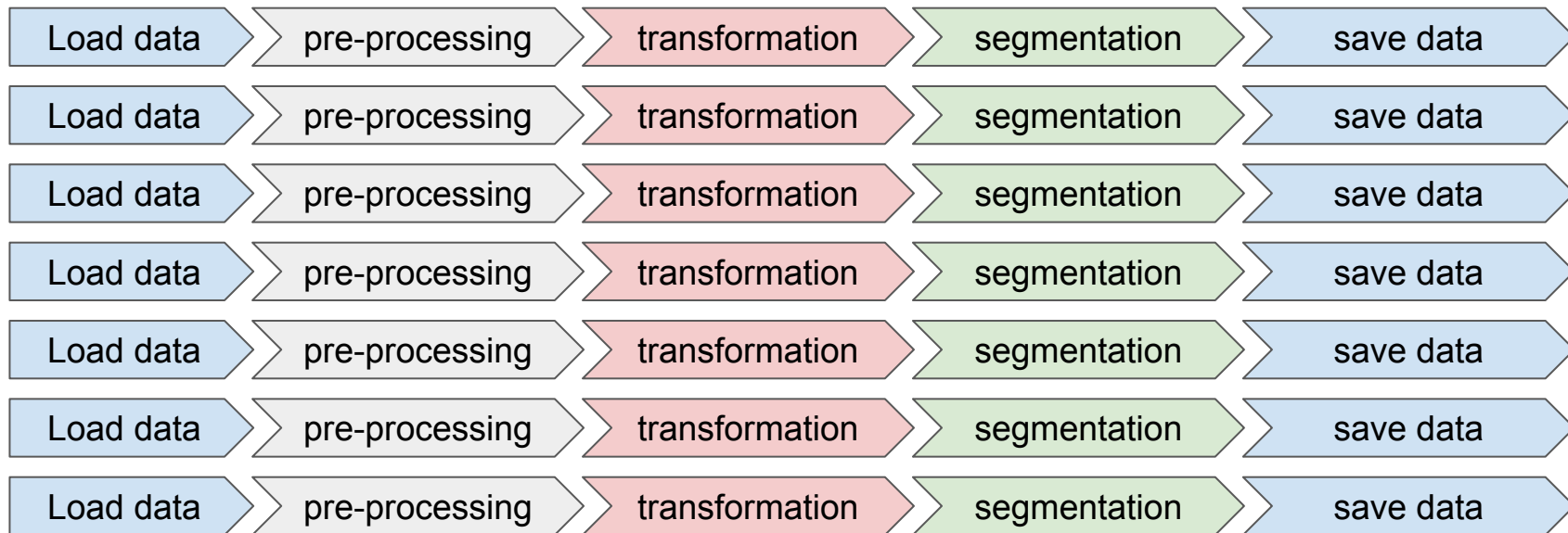
Memory management

Process per data units ...



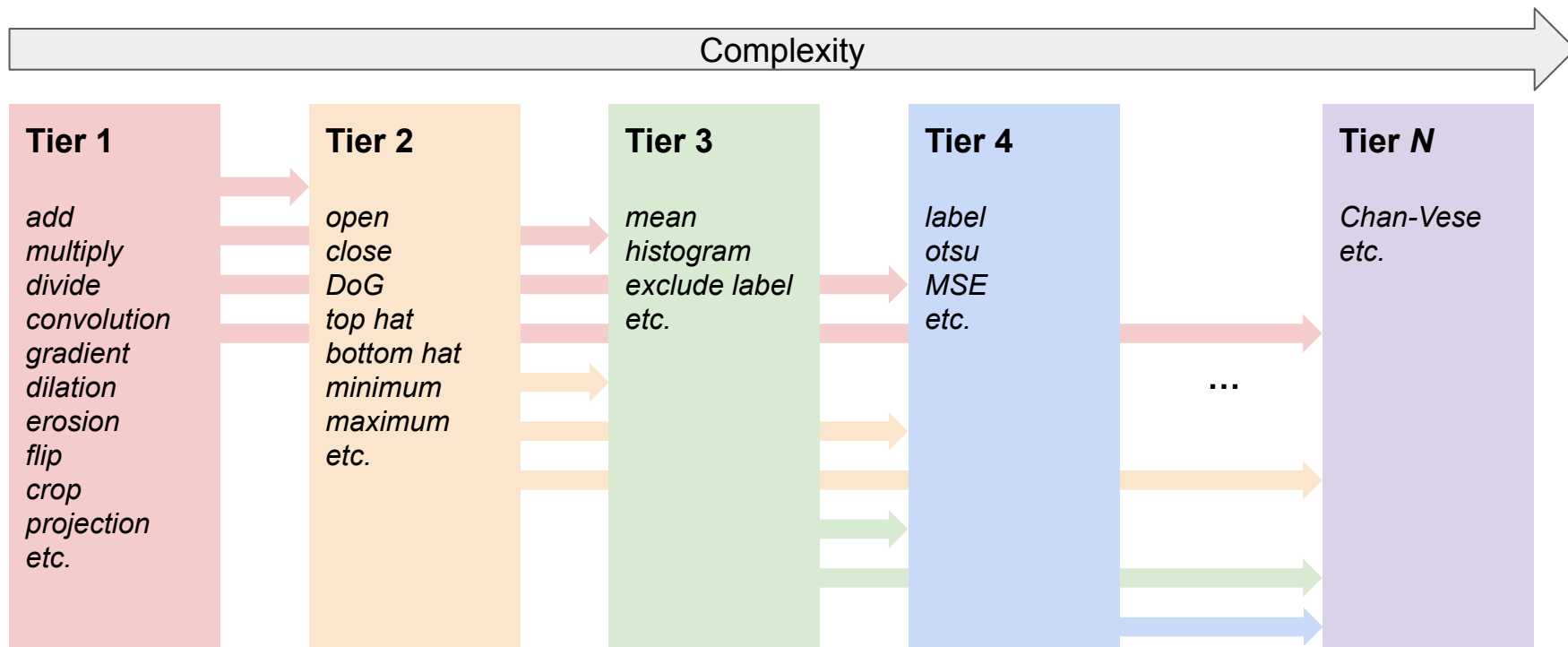
Memory management

Process per data units ... and distribute it between systems



Extensibility

Towards advance algorithm



Read The Doc

<https://clesperanto-doc.readthedocs.io>

And use the search bar

The screenshot displays the clesperanto documentation website. The top navigation bar includes the 'cle.' logo and a search bar. Below the search bar, a sidebar on the left lists navigation links: Introduction, clesperanto.J, pyclesperanto, and CLIC. The main content area is titled 'Welcome to clEsperanto!' and describes it as 'The GPU-accelerated Image Processing library for all!'. It features a list of links: Introduction (with sub-links: What's clEsperanto?, GPU acceleration, Ressources, References), clesperanto.J (with sub-links: Introduction, API Documentation), pyclesperanto (with sub-links: Installation, Usage, Tutorials & Ressources, Development, FAQs, API Documentation), and CLIC (with sub-links: Introduction, Library compilation, Library architecture, How to use the library, Contribute to the Library, API Documentation). A search bar is also present in the top navigation bar. Below the search bar, the search results are displayed, showing four pages matching the search query 'voronoi python'. The search results are listed as follows: pyclesperanto_tier2.extend_labeling_via_voronoi (Python function, in Filters), pyclesperanto_tier6.masked_voronoi_labeling (Python function, in Filters), pyclesperanto_tier6.voronoi_labeling (Python function, in Filters), and pyclesperanto_tier7.voronoi_otsu_labeling (Python function, in Filters).

Search Results

Search finished, found 4 pages matching the search query.

- pyclesperanto_tier2.extend_labeling_via_voronoi (Python function, in Filters)
- pyclesperanto_tier6.masked_voronoi_labeling (Python function, in Filters)
- pyclesperanto_tier6.voronoi_labeling (Python function, in Filters)
- pyclesperanto_tier7.voronoi_otsu_labeling (Python function, in Filters)

Practical 2: Image Processing

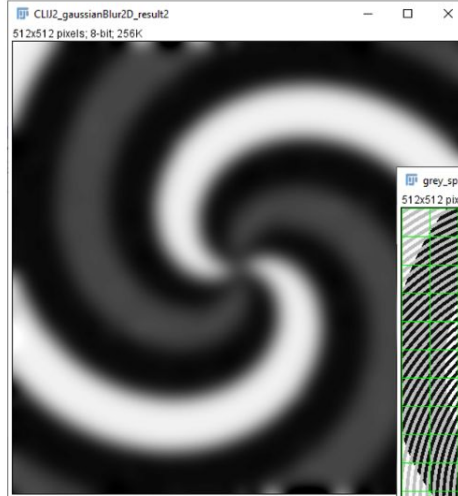
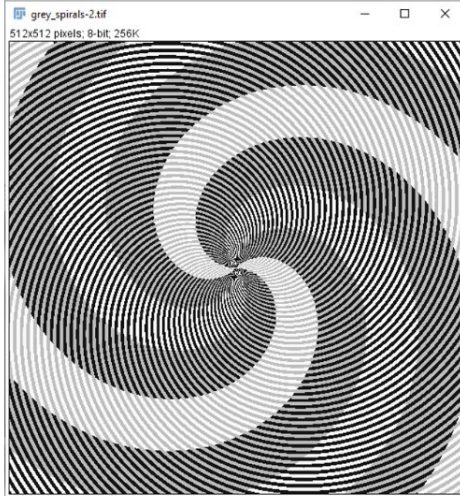
Adapt a Scikit-Image pipeline into a clEsperanto pipeline

Practical 3: Benchmarking

Test your GPU speed in different conditions ...

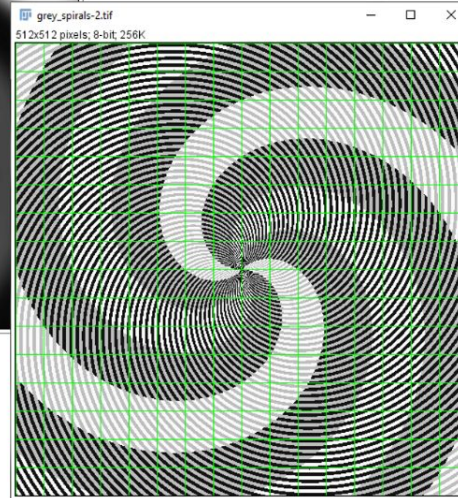
Practical 4: 3D Segmentation

Use clesperanto for 3D Nuclei segmentation



GPU are also limited by large data, same as CPU.

The answer is the same, partition the effort in small chunks and then try to fuse them back ...



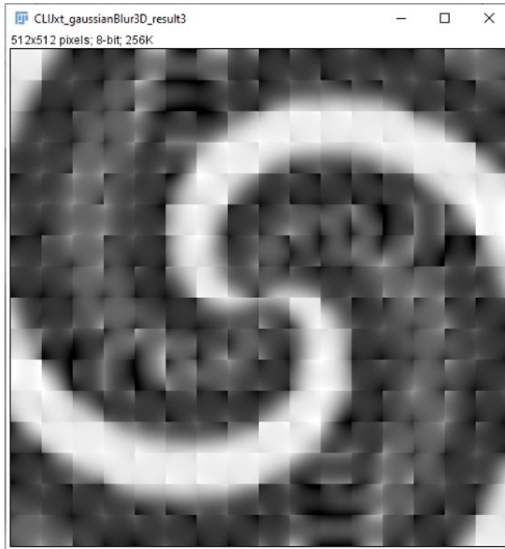
Tiling

Example: Gaussian blur (sigma = 20)

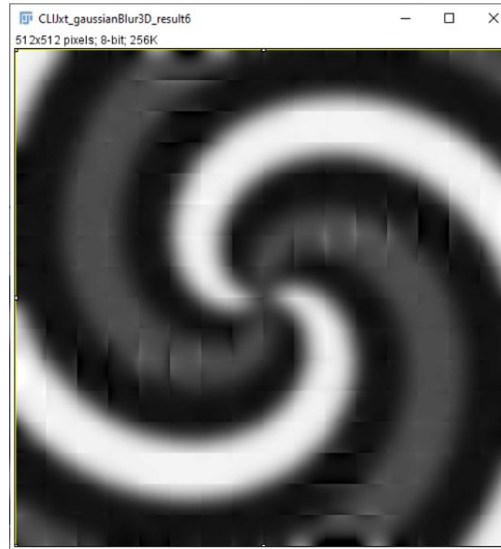
Solution: Process with overlapping tiles (size + margin)

Optimal margin size
depends on algorithm and
its parameters

Margin: 0 pixels



Margin: 10 pixels



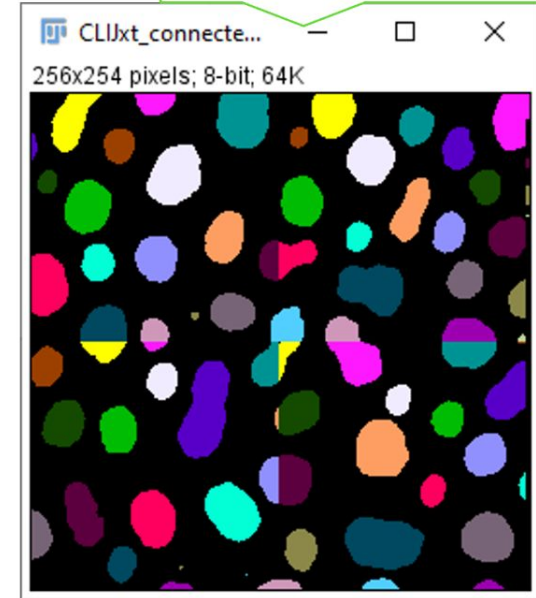
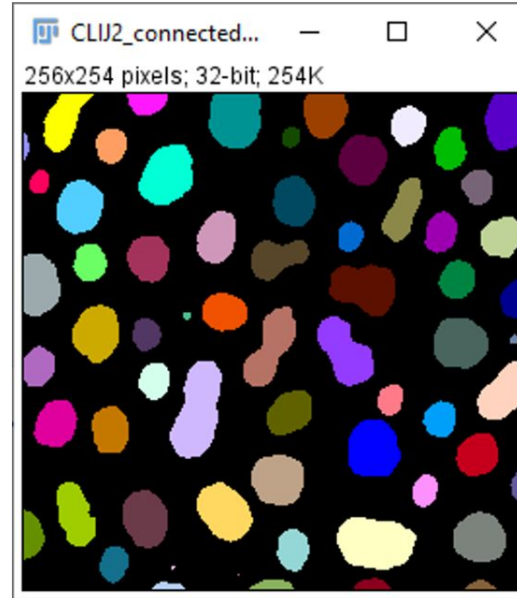
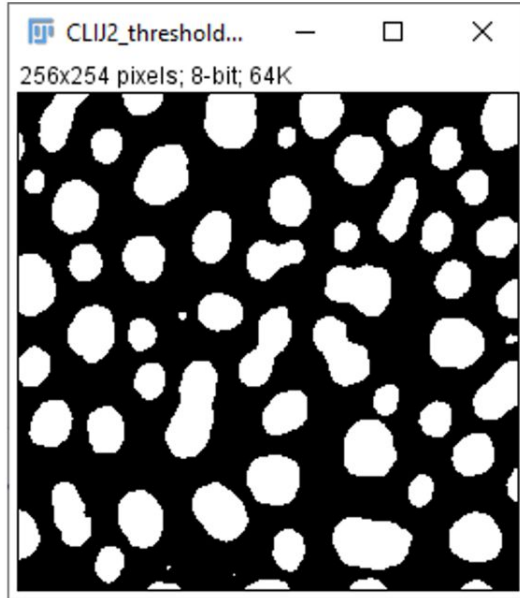
Margin: 20 pixels



Tiling

Some algorithms are hard to solve by processing tiles
Example: Connected component analysis

Checking which labels
touch and combine them
is feasible.

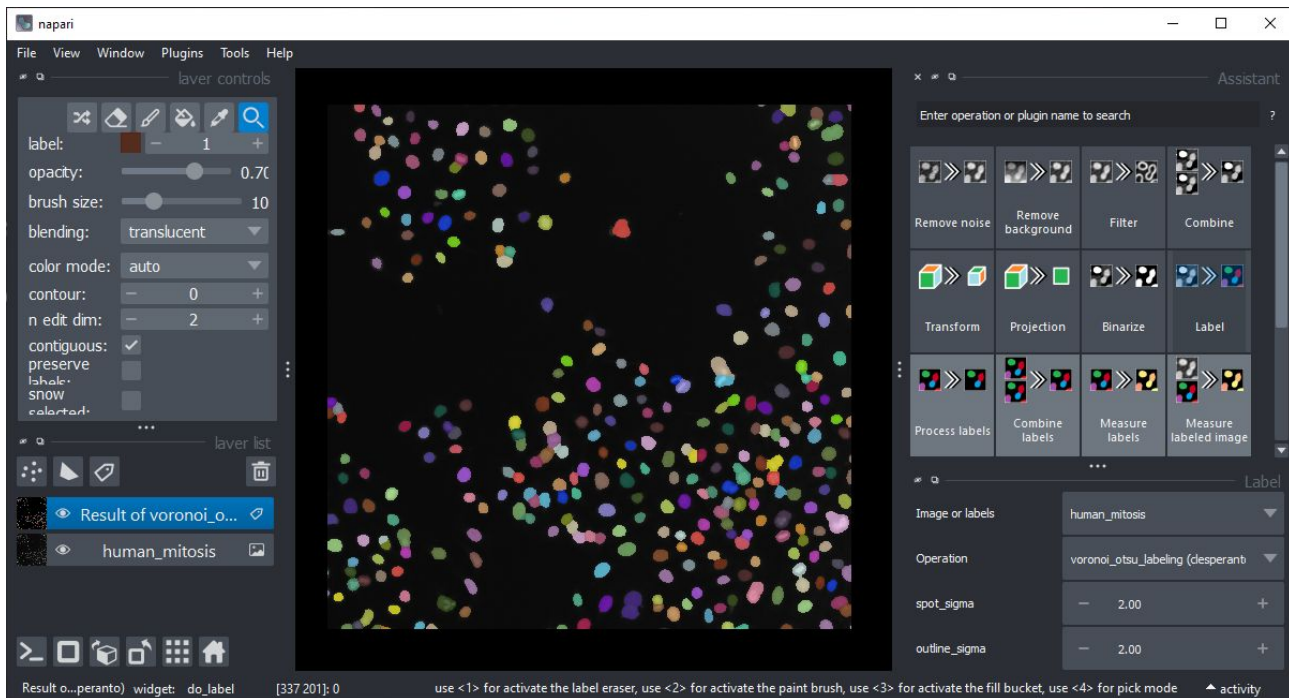


Practical 5: multi-gpu tile processing

napari assistant

www.napari-hub.org/plugins/napari-assistant

User-friendly bio-image analysis



Acknowledgments



Jean-Yves Tinevez



Marvin Albert



Minh-Son Phan



Cherif Latrech



Laura Xenard



Gaëlle Letort



<https://fiji.sc>



<https://napari.dev>



Robert Haase



Till Korten



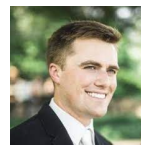
Johannes Soltweibel



Brian Northon



Pradeep Rajasekhar

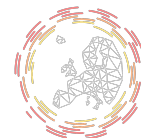


Mathew McCormick

Rest of the World



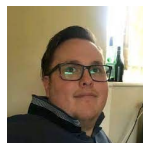
<https://image.sc>



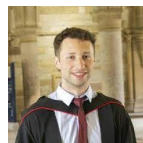
<https://eubias.org/NEUBIAS>



Martin Jones

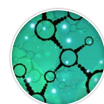


Jon Smith



Grahams Ross

Funding:



FRANCE
BIOIMAGING



CHAN
ZUCKERBERG
INITIATIVE

More Demos and Example : <https://github.com/cleEsperanto>

README.md

cleEsperanto is a GPU-accelerated image processing library that offers:

- Your favorite image processing functions on GPU
- A unified API accessible across various programming languages
- Broad compatibility with different hardware and systems

See the project [documentation](#) for more information.

Pinned

Customize pins

 [pyclesperanto](#) (Public)

GPU-accelerated Image Processing library for Python

 Jupyter Notebook  41  8

 [clesperanto.github.io](#) (Public)

cleEsperanto - GPU-accelerated image processing across languages and platforms

 11  2